

Sistemas Distribuidos

GRADO EN INGENIERÍA INFORMÁTICA



Práctica

Servicio de envío de mensajes

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	David Sierra
NIA:	100471913
Correo electrónico:	100471913@alumnos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Grado en Ingeniería Informática

Nombre:	Juan Luis del Valle Caetano
NIA:	100454201
Correo electrónico:	100454201@alumbos.uc3m.es
Grupo de clase:	80
Titulación cursada:	Doble grado en Ingeniería Informática y ADE

1. Descripción del Sistema	2
2. Descripción del código	3
2.a Servidor de mensajería (server.c)	3
2b. Cliente python (client.py)	4
2.c Servidor web (web-service.py)	5
2.d Servidor RPC (log_rpc.x + log_rpc_server_impl.c)	6
3. Ejecución y compilación	6
4. Batería de pruebas	8
4.1 Registro y baja de usuarios	8
4.2 Conexión y desconexión	9
4.3 Envío de mensajes	9
4.4 SENDATTACH y transferencia de ficheros	10
6.5 Users	11
4.6. Concurrencia	12
4.7 Servidor RPC	12
5. Conclusiones	13

1. Descripción del Sistema

La práctica implementa un servicio de mensajería distribuido similar a WhatsApp, compuesto por cuatro componentes que se comunican mediante distintas tecnologías:

Servicio de mensajería (`server.c`) — servidor concurrente multihilo en C que gestiona el registro de usuarios, sus conexiones y el envío de mensajes. Utiliza sockets TCP para comunicarse con los clientes. Actúa también como cliente RPC para registrar cada operación en el servicio de log.

Cliente (`client.py`) — cliente en Python que permite a los usuarios registrarse, conectarse, enviar mensajes y transferir ficheros. Mantiene un hilo de escucha permanente para recibir mensajes entrantes del servidor. Antes de enviar cada mensaje, lo normaliza a través del servicio web local.

Servicio web (`web_service.py`) — servidor HTTP en Python que expone el endpoint `POST /normalize`. Elimina los espacios en blanco repetidos de los mensajes para que las palabras queden separadas por un único espacio.

Servidor RPC (`log_rpc.x` + `log_rpc_server_impl.c`) — servidor ONC-RPC en C que recibe y registra por pantalla cada operación realizada por los usuarios del sistema. El servidor de mensajería se conecta a él al arrancar, leyendo su dirección IP de la variable de entorno `LOG_RPC_IP`.

2. Descripción del código

2.a Servidor de mensajería (`server.c`)

El servidor mantiene en memoria dos listas enlazadas dinámicas. La estructura `Usuario` almacena el nombre, estado de conexión, IP, puerto de escucha, contador de mensajes y lista de mensajes pendientes. La estructura `Mensaje` almacena el identificador, remitente, texto y nombre de fichero adjunto (vacío si no aplica). Ambas listas están protegidas por un mutex POSIX para garantizar acceso exclusivo entre hilos.

Funciones principales:

buscar_usuario — recorre la lista de usuarios buscando por nombre. Requiere el mutex bloqueado.

agregar_usuario — reserva memoria con **malloc** e inserta un nuevo usuario al principio de la lista.

eliminar_usuario — borra un usuario de la lista liberando su memoria y la de todos sus mensajes pendientes.

agregar_mensaje — añade un mensaje al final de la lista de mensajes pendientes de un usuario, asignando un identificador incremental de tipo **unsigned int**.

enviar_cadena / **recibir_cadena** / **enviar_byte** — utilidades de comunicación por socket. **recibir_cadena** lee byte a byte hasta encontrar el carácter nulo para evitar problemas de fragmentación de TCP.

conectar_a_cliente — abre una conexión TCP al hilo de escucha de un cliente dado su IP y puerto. La usa el servidor para entregar mensajes y enviar confirmaciones.

entregar_mensaje — implementa el protocolo de entrega servidor→cliente (sección 8.6 del enunciado). Envía la operación **SEND_MESSAGE** o **SEND_MESSAGE_ATTACH** según si el mensaje lleva fichero adjunto o no.

notificar_remitente — notifica al remitente que su mensaje fue entregado, enviando **SEND_MESS_ACK** o **SEND_MESS_ATTACH_ACK** según el tipo de mensaje.

op_register / **op_unregister** — gestionan el alta y baja de usuarios. Comprueban existencia del usuario y responden con un byte de resultado.

op_connect — conecta un usuario, guarda su IP y puerto, y entrega todos los mensajes pendientes uno a uno.

op_disconnect — desconecta un usuario borrando su IP y puerto.

op_send / **op_sendattach** — procesan el envío de mensajes con y sin fichero adjunto. Almacenan el mensaje en la lista de pendientes, responden al remitente con el identificador asignado, e intentan entrega inmediata si el destinatario está conectado.

op_users — devuelve la lista de usuarios conectados en formato **nombre :: IP :: puerto** para que los clientes puedan realizar transferencias de ficheros directas.

atender_cliente — función ejecutada por cada hilo. Lee la operación recibida y llama a la función correspondiente.

rpc_log — envía al servidor RPC el nombre del usuario, la operación realizada y el fichero adjunto si aplica. Solo actúa si **LOG_RPC_IP** está definida.

main — parsea el puerto, inicializa la conexión RPC si procede, configura señales, crea el socket y entra en el bucle principal de **accept** + **pthread_create**.

2b. Cliente python (*client.py*)

Atributos de clase:

_server y **_port** — dirección y puerto del servidor de mensajería. **_connected_user** — nombre del usuario actualmente conectado. **_listen_sock** y **_listen_thread** — socket e hilo de escucha de mensajes entrantes. **_usuarios_conectados** — diccionario **{nombre: (ip, puerto)}** que se actualiza con cada llamada a **users**. **_web_port** — puerto del servicio web local de normalización.

Helpers:

`_enviar_cadena` / `_recibir_cadena` / `_recibir_byte` — utilidades de comunicación equivalentes a las del servidor. `_conectar` — crea y devuelve un socket TCP conectado al servidor. `_normalizar_mensaje` — envía el mensaje al servicio web local mediante una petición HTTP POST y devuelve el mensaje normalizado. Si el servicio no está disponible, devuelve el mensaje sin modificar.

`_hilo_escucha` — hilo daemon que acepta conexiones entrantes en el puerto de escucha del cliente y gestiona cuatro tipos de mensajes: `SEND_MESSAGE` (mensaje simple), `SEND_MESS_ACK` (confirmación de entrega simple), `SEND_MESSAGE_ATTACH` (mensaje con fichero adjunto), `SEND_MESS_ATTACH_ACK` (confirmación con fichero), y `GET_FILE` (petición de transferencia de fichero P2P).

Métodos de la API:

`register` / `unregister` — alta y baja en el sistema siguiendo el protocolo de la sección 8.1 y 8.2.

`connect` — busca un puerto libre, crea el hilo de escucha, envía la solicitud de conexión con nombre y puerto, y gestiona los mensajes pendientes que entrega el servidor.

`disconnect` — envía la desconexión al servidor y cierra el socket de escucha parando el hilo.

`users` — solicita la lista de usuarios conectados, parsea las entradas `nombre :: IP :: puerto` y actualiza `_usuarios_conectados`.

`send` / `sendAttach` — envían mensajes con y sin fichero adjunto. Normalizan el mensaje a través del servicio web antes de enviarlo.

`getfile` — busca la IP y puerto del usuario remoto en `_usuarios_conectados`, se conecta directamente a su hilo de escucha, envía `GET_FILE` y recibe el contenido del fichero guardándolo localmente.

2.c Servidor web (*web-service.py*)

Servidor HTTP implementado con `http.server` de Python. Expone el endpoint `POST /normalize` que acepta un JSON con el campo `message`, aplica una expresión regular para colapsar múltiples espacios en uno y eliminar espacios al inicio y al final, y devuelve el mensaje normalizado en formato JSON. Se despliega en la máquina local del cliente y se arranca con `python3 web_service.py -p <puerto>`.

2.d Servidor RPC (*log_rpc.x + log_rpc_server_impl.c*)

Interfaz (*log_rpc.x*): define la estructura *LogPetición* con tres cadenas — *usuario*, *operacion* y *fichero* — y el procedimiento *log_operacion* dentro del programa *LOG_PROG* (número *0x30000099*). Se eligió una única estructura con tres campos para cubrir todos los casos con un solo procedimiento, enviando el campo *fichero* vacío cuando la operación no es *SENDATTACH*.

Implementación (*log_rpc_server_impl.c*): implementa *log_operacion_1_svc* que imprime por pantalla el usuario y la operación recibidos. Para *SENDATTACH* también muestra el nombre del fichero adjunto. El servidor se genera a partir de los ficheros producidos por *rpcgen* y se registra automáticamente en *rpcbind*.

3. Ejecución y compilación

Requisitos previos:

- GCC con soporte para C99 o superior
 - *rpcbind* instalado y en ejecución (*sudo systemctl restart rpcbind*)
 - Python 3.8 o superior
 - librería *libtirpc-dev* instalada (*sudo apt install libtirpc-dev*)
-

Compilación:

Desde el directorio de la práctica ejecutar:

```
Shell  
make all
```

Esto genera en orden los ficheros RPC a partir de *log_rpc.x* mediante *rpcgen*, y compila los ejecutables *server* y *log_server*. Para limpiar todos los ficheros generados:

```
Shell  
make clean
```

Ejecución:

Se distinguen dos modos según las funcionalidades que se quieran probar.

Modo completo (partes 1 y 2):

```
Shell
# Terminal 1 - Servidor RPC

./log_server

# Terminal 2 - Servidor de mensajería con RPC

export LOG_RPC_IP=localhost

./server -p 8888

# Terminal 3 - Servicio web (máquina del cliente)

python3 web_service.py -p 8080

# Terminal 4 - Cliente

python3 client.py -s localhost -p 8888 -w 8080
```

Modo básico (solo parte 1):

```
Shell
# Terminal 1

./server -p 8888

# Terminal 2

python3 client.py -s localhost -p 8888
```

Ejecución en entornos distribuidos:

El sistema es compatible con clientes y servidor ejecutando en máquinas con IPs distintas. Basta con indicar la IP del servidor en el parámetro `-s` del cliente y definir `LOG_RPC_IP` con la IP de la máquina donde corre el servidor RPC.

4. Batería de pruebas

Nota sobre la ejecución de las pruebas:

Para ejecutar las pruebas es necesario tener al menos dos terminales abiertas. El servidor se arranca en una terminal con `./server -p 8888` y no acepta comandos por teclado — solo muestra mensajes de log. Los comandos se introducen en el cliente, que se arranca en otra terminal con `python3 client.py -s localhost -p 8888`.

En las tablas de pruebas se usa la siguiente notación:

- `c>` — salida o entrada del **cliente**
- `s>` — salida del **servidor**
- **Terminal 1, Terminal 2...** — indica en qué instancia de cliente se ejecuta cada comando cuando hay varios clientes simultáneos

4.1 Registro y baja de usuarios

Prueba	Entrada	Salida esperada	Estado
Registro correcto	Terminal 1: <code>REGISTER juan</code>	<code>c> REGISTER OK</code>	Correcta
Registro usuario existente	Terminal 1: <code>REGISTER juan</code> → <code>REGISTER juan</code>	<code>c> USERNAME IN USE</code>	Correcta
Baja de usuario existente	Terminal 1: <code>REGISTER juan</code> → <code>UNREGISTER juan</code>	<code>c> UNREGISTER OK</code>	Correcta
Baja de usuario inexistente	Terminal 1: <code>UNREGISTER noexiste</code>	<code>c> USER DOES NOT EXIST</code>	Correcta
Baja borra mensajes pendientes	Terminal 1: <code>REGISTER juan</code> → <code>REGISTER david</code> → <code>CONNECT juan</code> → <code>SEND david hola /</code>	Terminal 2 no recibe ningún mensaje	Correcta

	Terminal 2: CONNECT david (después de UNREGISTER y REGISTER david)		
--	--	--	--

4.2 Conexión y desconexión

Prueba	Entrada	Salida esperada	Estado
Conexión correcta	Terminal 1: REGISTER juan → CONNECT juan	c> CONNECT OK / s> CONNECT juan OK	Correcta
Conexión de usuario inexistente	Terminal 1: CONNECT noexiste	c> CONNECT FAIL, USER DOES NOT EXIST	Correcta
Doble conexión del mismo usuario	Terminal 1: REGISTER juan → CONNECT juan → CONNECT juan	c> USER ALREADY CONNECTED	Correcta
Desconexión correcta	Terminal 1: REGISTER juan → CONNECT juan → DISCONNECT juan	c> DISCONNECT OK / s> DISCONNECT juan OK	Correcta
Desconexión de usuario no conectado	Terminal 1: REGISTER juan → DISCONNECT juan	c> DISCONNECT FAIL, USER NOT CONNECTED	Correcta
Desconexión de usuario inexistente	Terminal 1: DISCONNECT noexiste	c> DISCONNECT FAIL, USER DOES NOT EXIST	Correcta

4.3 Envío de mensajes

Para la última prueba necesitas una terminal y ejecutar `python3 web_service.py -p 8080`.

Prueba	Entrada	Salida esperada	Estado
Entrega inmediata	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david → SEND juan hola	Terminal 1: s> MESSAGE 1 FROM david / Terminal 2: c> SEND MESSAGE 1 OK	Correcta

Mensaje almacenado y entregado al conectarse	Terminal 1: REGISTER juan / Terminal 2: REGISTER david → CONNECT david → SEND juan pendiente / Terminal 1: CONNECT juan	Terminal 1 recibe el mensaje al conectarse	Correcta
Envío a usuario inexistente	Terminal 1: REGISTER juan → CONNECT juan → SEND noexiste hola	c> SEND FAIL, USER DOES NOT EXIST	Correcta
Múltiples mensajes pendientes entregados en orden	Terminal 1: REGISTER juan / Terminal 2: REGISTER david → CONNECT david → SEND juan msg1 → SEND juan msg2 → SEND juan msg3 / Terminal 1: CONNECT juan	Terminal 1 recibe msg1, msg2, msg3 en ese orden	Correcta
Normalización de espacios	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david → SEND juan hola como estas	Terminal 1 recibe hola como estas	Correcta

4.4 SENDATTACH y transferencia de ficheros

Primero crea un fichero de prueba:

Shell

```
echo "contenido de prueba" > /tmp/prueba.txt
```

Prueba	Entrada	Salida esperada	Estado
SENDATTACH con entrega inmediata	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david → SENDATTACH juan /tmp/prueba.txt mensaje	Terminal 1: c> MESSAGE 1 FROM david / FILE /tmp/prueba.txt / Terminal 2: c> SENDATTACH MESSAGE 1 /tmp/prueba.txt OK	Correcto
SENDATTACH a usuario	Terminal 1: REGISTER juan →	c> SENDATTACH	correcto

inexistente	CONNECT juan → SENDATTACH noexiste /tmp/prueba.txt mensaje	FAIL, USER DOES NOT EXIST	
SENDATTACH almacenado y entregado al conectarse	Terminal 1: REGISTER juan / Terminal 2: REGISTER david → CONNECT david → SENDATTACH juan /tmp/prueba.txt mensaje / Terminal 1: CONNECT juan	Terminal 1 recibe el mensaje y nombre de fichero al conectarse	correcto
GETFILE transferencia P2P	Tras recibir SENDATTACH: Terminal 1: GETFILE david /tmp/prueba.txt /tmp/copia.txt	c> CONNECTED USERS (2 users connected) OK david :: 127.0.0.1 :: 40387 juan :: 127.0.0.1 :: 55071 c> FILE TRANSFER OK	Correcto
GETFILE a usuario desconectado	Terminal 2: DISCONNECT david / Terminal 1: GETFILE david /tmp/prueba.txt /tmp/copia.txt	c> FILE TRANSFER FAILED, user not connected.	Correcto

6.5 Users

Prueba	Entrada	Salida esperada	Estado
Listar usuarios conectados	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david → USERS	c> CONNECTED USERS (2 users connected) OK / juan / david	Correcto
USERS sin estar conectado	Terminal 1: REGISTER juan → USERS	c> CONNECTED	correcto

		USERS FAIL, USER IS NOT CONNECTED	
USERS devuelve IP y puerto para GETFILE	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david → USERS → SENDATTACH juan /tmp/prueba.txt msg / Terminal 1: GETFILE david /tmp/prueba.txt /tmp/copia.txt	c> FILE TRANSFER OK — verifica que USERS actualizó correctamente la tabla interna de IPs	correcto

4.6. Concurrency

Prueba	Entrada	Salida esperada	Estado
Múltiples clientes simultáneos	Terminal 1: REGISTER juan → CONNECT juan / Terminal 2: REGISTER david → CONNECT david / Terminal 3: REGISTER maria → CONNECT maria → SEND juan hola → SEND david hola	Todos los mensajes llegan correctamente sin corrupción de datos	Correcto

4.7 Servidor RPC

Para estas pruebas necesitas arrancar también el servidor RPC:

Terminal 1 — Servidor RPC:

```
Shell
sudo systemctl restart rpcbind
./log_server
```

Terminal 2 — Servidor de mensajería:

```
Shell
export LOG_RPC_IP=localhost
./server -p 8888
```

Terminal 3 — Cliente:

Shell

```
python3 client.py -s localhost -p 8888
```

Prueba	Entrada	Salida esperada	Estado
Registro de operaciones en el log	Terminal 3: REGISTER juan → CONNECT juan → SEND juan hola → DISCONNECT juan → UNREGISTER juan	Terminal 1 muestra: juan REGISTER / juan CONNECT / juan SEND / juan DISCONNECT / juan UNREGISTER	correcto
SENDATTACH registrado con fichero	Terminal 3: REGISTER juan → CONNECT juan → SENDATTACH juan /tmp/prueba.tx t msg	Terminal 1 muestra: juan SENDATTACH /tmp/prueba.tx t	correcto
Servidor funciona sin RPC configurado	Arrancar ./server -p 8888 sin definir LOG_RPC_IP / Terminal 3: REGISTER juan → CONNECT juan → SEND juan hola	Todas las operaciones funcionan correctamente sin RPC	correcto

5. Conclusiones

La práctica ha permitido profundizar en el diseño e implementación de una aplicación distribuida real, combinando varias tecnologías estudiadas durante el curso: sockets TCP, servicios web y ONC-RPC.

Aspectos técnicos destacados:

El principal reto de la parte 1 fue gestionar correctamente la concurrencia en el servidor. El uso de mutex POSIX para proteger la lista de usuarios y la elección de hilos bajo demanda en modo detached garantizan que múltiples clientes puedan operar simultáneamente sin condiciones de carrera ni fugas de memoria.

La comunicación servidor→cliente para la entrega de mensajes supuso un cambio de paradigma respecto a los ejercicios evaluables: el servidor actúa también como cliente TCP, conectándose al hilo de escucha de cada usuario. Esto requirió un diseño cuidadoso para evitar interbloqueos al acceder a la lista de usuarios desde múltiples hilos.

En la parte 2, la transferencia de ficheros P2P entre clientes resultó especialmente interesante al no pasar por el servidor, comunicándose directamente los dos hilos de escucha de los clientes implicados.

La integración del servicio web de normalización se diseñó de forma no bloqueante — si el servicio no está disponible, el mensaje se envía sin normalizar, garantizando que el sistema sigue funcionando.

Problemas encontrados:

El principal problema durante el desarrollo fue la ausencia de la librería `libtirpc` en el entorno de compilación, necesaria para el soporte de ONC-RPC en sistemas Linux modernos. Se resolvió instalando el paquete `libtirpc-dev` y añadiendo los flags `-I/usr/include/tirpc` y `-ltirpc` al Makefile. Adicionalmente, el código generado automáticamente por `rpcgen` producía un warning sobre una variable no utilizada en `log_rpc_xdr.c`, que se suprimió compilando ese fichero por separado con `-Wno-unused-variable`.

Opinión personal:

La práctica ha sido una buena forma de consolidar los conocimientos adquiridos en los ejercicios evaluables aplicándolos a un sistema más complejo y realista. La combinación de C y Python en el mismo proyecto, junto con la integración de tres tecnologías de comunicación distintas, refleja bien la diversidad de entornos que se pueden encontrar en un sistema distribuido real.